

# Project Design Phase-II

## Technology Stack (Architecture & Stack)

|              |                           |               |                    |
|--------------|---------------------------|---------------|--------------------|
| Date         | 14 June 2026              | Team ID       | N/A (Solo Project) |
| Project Name | Network Anomaly Detection | Maximum Marks | 4 Marks            |

### Technical Architecture:

The deliverable includes the architectural diagram below and the information as per Table 1 & Table 2. The architecture shows the full pipeline of the Network Anomaly Detection system running on a local development machine, with CICIDS 2017 as the external data source.

1. All processes are included as application logic / technology blocks.
2. Infrastructural demarcation is shown: Local Machine vs External Data Source vs Output Layer.
3. External interface indicated: CICIDS 2017 Dataset (CSV files from University of New Brunswick).
4. Data storage components indicated: Feature Store, Model Storage (.pkl), Results Store (CSV).
5. Machine learning model interface indicated: Isolation Forest via sklearn with predict() and score\_samples().

### Technical Architecture Diagram

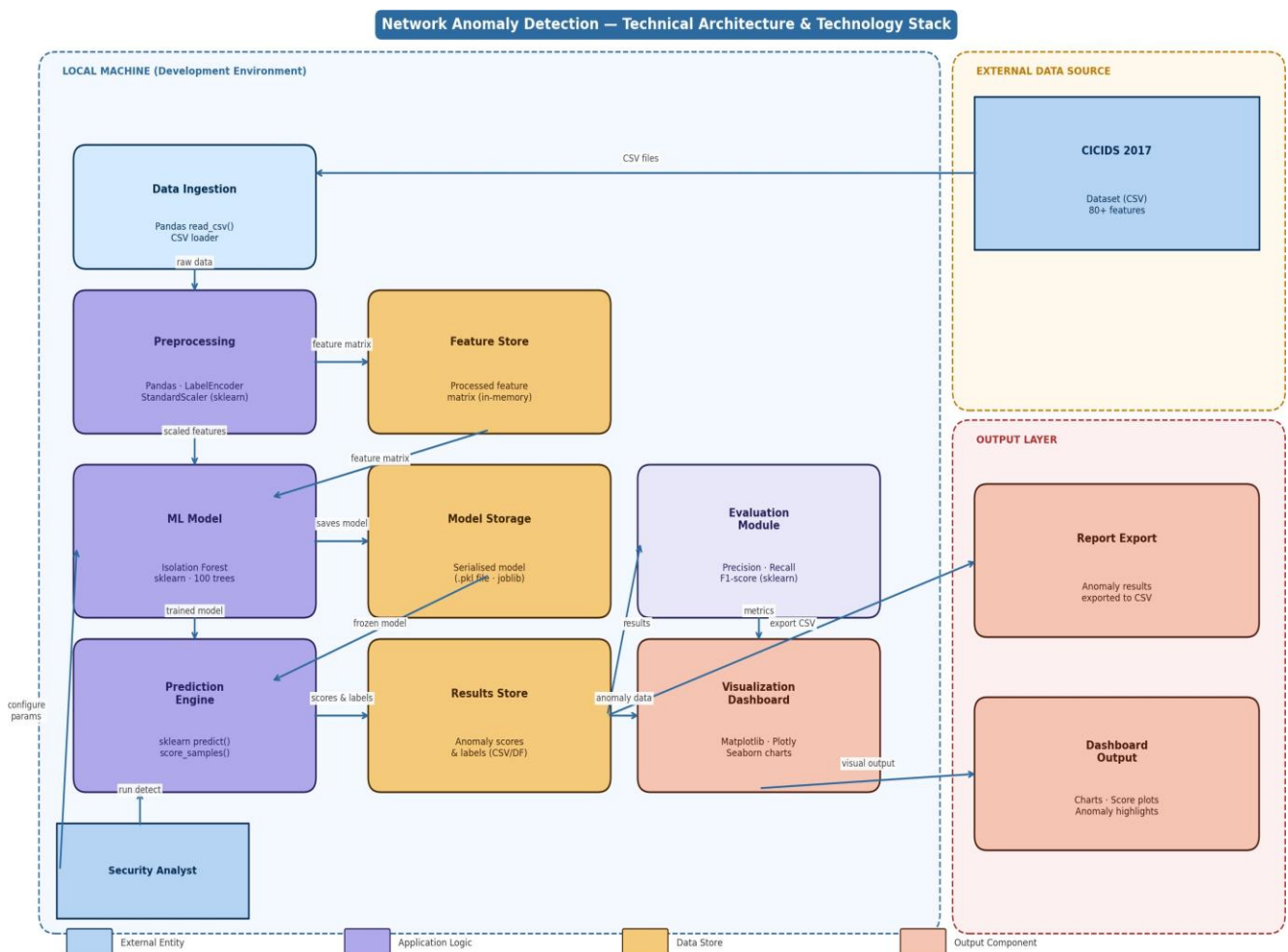


Figure 1: Technical architecture of the Network Anomaly Detection system showing local pipeline, external data source (CICIDS 2017), data stores, ML model interface, and output layer

### Table 1: Components & Technologies

| S.No | Component                               | Description                                                                                          | Technology                                                                               |
|------|-----------------------------------------|------------------------------------------------------------------------------------------------------|------------------------------------------------------------------------------------------|
| 1    | User Interface                          | How the security analyst interacts with the system — command-line interface and output dashboard     | Python CLI · Matplotlib · Plotly (HTML charts)                                           |
| 2    | Application Logic 1 (Data Ingestion)    | Loads CICIDS 2017 CSV files into memory for processing                                               | Python · Pandas (read_csv)                                                               |
| 3    | Application Logic 2 (Preprocessing)     | Cleans data, encodes categorical features, and normalises feature ranges                             | Pandas · sklearn LabelEncoder · StandardScaler                                           |
| 4    | Application Logic 3 (ML Model Training) | Trains Isolation Forest model on processed traffic data using 100 decision trees                     | sklearn IsolationForest (n_estimators=100, contamination=0.05)                           |
| 5    | Application Logic 4 (Prediction Engine) | Runs anomaly detection on unseen traffic; produces anomaly scores and class labels                   | sklearn predict() · score_samples()                                                      |
| 6    | Application Logic 5 (Evaluation Module) | Computes model performance metrics — precision, recall, F1-score using ground truth labels           | sklearn classification_report · confusion_matrix                                         |
| 7    | Database / File Storage                 | Stores raw CSV data, processed feature matrix (in-memory), trained model object, and anomaly results | Local Filesystem · Pandas DataFrame · joblib (.pkl model file) · CSV output              |
| 8    | External Data Source                    | CICIDS 2017 dataset — source of labelled network traffic flows (normal + attack types)               | CSV files · University of New Brunswick (publicly available dataset)                     |
| 9    | Machine Learning Model                  | Isolation Forest — unsupervised anomaly detection via random feature-based tree splitting            | sklearn.ensemble.IsolationForest Python 3.x                                              |
| 10   | Visualization Dashboard                 | Plots anomaly score distributions, flagged traffic flows, and feature contribution charts            | Matplotlib · Seaborn · Plotly                                                            |
| 11   | Infrastructure (Local)                  | All components run on a local development machine. No cloud deployment required for MVP.             | Local Machine · Python 3.10+ Anaconda / venv environment Jupyter Notebook or .py scripts |

**Table 2: Application Characteristics**

| S.No | Characteristics          | Description                                                                                                                                                                                                                                                                                   | Technology                                                                                                  |
|------|--------------------------|-----------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|-------------------------------------------------------------------------------------------------------------|
| 1    | Open-Source Frameworks   | All libraries used are fully open-source with active community support and extensive documentation                                                                                                                                                                                            | Python 3.x · Pandas · NumPy · scikit-learn · Matplotlib · Seaborn · Plotly · joblib                         |
| 2    | Security Implementations | No user authentication required for solo academic project. Dataset access is read-only. Model file (.pkl) is stored locally with filesystem-level access control.                                                                                                                             | Local filesystem permissions · Read-only dataset access · No sensitive data stored                          |
| 3    | Scalable Architecture    | The modular pipeline design allows each component (ingestion, preprocessing, model, visualization) to be replaced or upgraded independently. The Isolation Forest model can be retrained on new data without redesigning the pipeline. Can be containerised with Docker for cloud deployment. | Modular Python scripts · Docker-ready · sklearn pipeline API · joblib model serialisation                   |
| 4    | Availability             | As a local MVP, the system runs on demand. For production deployment, the pipeline can be scheduled using cron jobs or Apache Airflow for automated periodic retraining and detection.                                                                                                        | Local execution · Schedulable via cron · Apache Airflow (future) · Docker containers (future)               |
| 5    | Performance              | Isolation Forest training on CICIDS 2017 (~2.8M rows) completes in under 2 minutes on a standard laptop. Prediction (inference) on test data is near-instantaneous. In-memory Pandas DataFrames are used to avoid repeated disk I/O.                                                          | In-memory Pandas DataFrames · sklearn vectorised operations · NumPy array processing · joblib parallel jobs |

## References:

- <https://scikit-learn.org/stable/modules/generated/sklearn.ensemble.IsolationForest.html>
- <https://www.unb.ca/cic/datasets/ids-2017.html>
- <https://c4model.com/>
- <https://www.ibm.com/cloud/architecture>